CREATING A MOTORBIKES SALES DATABASE USING SQLITE SOFTWARE AND PYTHON

NAME: CHRISTIAN OWOBU

STUDENT NUMBER: 22017251

ABSTRACT

This report using Python code demonstrates a method for generating a dataset that simulates a sales database for motorbikes. This database includes information about motorbike sales, the different motorbike models, and the brands. It utilizes libraries such as pandas for data manipulation and NumPy for numerical operations, with a focus on creating a realistic representation of a sales dataset while maintaining reproducibility through a fixed random seed.

INTRODUCTION

The objective is to create a multi-table database of a motorbike sales structure that not only adheres to the principles of database normalisation but also incorporate realistic data constraints and relationships. The database is structured around 3 tables namely Brand table, Motorbike table and Sales table with combination of 1000 rows and 8 columns. These tables include the nominal, ordinal, interval, and ratio data using foreign and composite keys to get meaningful relationships. By doing so, it aims to stimulate a real -world scenario that is both intricate and informative.

1. Generation of Data:

The data was generated through a systematic approach using **pandas** and **NumPy**:

pandas.DataFrame objects were created, populated with randomly generated data for sales, motorbikes, and brands, ensuring a varied dataset resembling real-world data. The random seed was set for reproducibility, ensuring the same random data is generated each time the script runs.

Numerical data such as **Sale_Price** and **Mileage** were generated with specified ranges.

Categorical data such as **Brand** and **Condition** were sampled from predefined lists.

A unique identifier for each motorbike was created by combining brand, condition, and manufacture year.

Creating a Database for Motorbike Sales

In [102]:

```
#importing necessary libraries

import pandas as pd
import numpy as np

# Set seed for reproducibility
np.random.seed(0)

# Generate random data for motorbike sales
num_rows = 1000 # Number of rows
brands = ['Yamaha', 'Honda', 'Suzuki', 'Kawasaki', 'BMW', 'Ducati', 'Harley-Davidson', 'KTM']
conditions = ['New', 'Like New', 'Good', 'Fair', 'Poor']
sale_months = ['January', 'February', 'March', 'April', 'May', 'June', 'July', 'August', 'September', 'October', 'November',

# Create DataFrame for motorbike sales
df_motorbike_sales = pd.DataFrame({
    'Sale_ID': np.arange(1, num_rows + 1),
    'Brand': np.random.choice(brands, size=num_rows),
    'Condition': np.random.choice(conditions, size=num_rows, p=[0.2, 0.2, 0.3, 0.2, 0.1]),
    'Sale_Month': np.random.choice(sale_months, size=num_rows),
    'Sale_Price': np.random.uniform(1500, 25000, size=num_rows).round(2),
    'Mileage': np.random.uniform(0, 50000, size=num_rows).round(),
    'Manufacture_Year': np.random.randint(2000, 2023, size=num_rows)
})

df_motorbike_sales
```

Out[102]:

	Sale_ID	Brand	Condition	Sale_Month	Sale_Price	Mileage	Manufacture_Year
0	1	BMW	Like New	June	22851.79	28290.0	2016
1	2	KTM	Like New	February	16975.79	23660.0	2011
2	3	Ducati	Good	August	12201.89	22735.0	2008
3	4	Yamaha	Fair	July	12471.26	11628.0	2005
4	5	Kawasaki	Like New	February	6473.48	13692.0	2015
...	...	...	...	...	...	...	...
995	996	BMW	New	September	19289.71	14753.0	2013
996	997	Yamaha	New	August	9268.90	16213.0	2003
997	998	Suzuki	New	April	11270.91	1119.0	2012
998	999	Honda	New	June	4276.71	14983.0	2019
999	1000	Yamaha	Like New	October	20410.10	47085.0	2019

In [115]:
```
# Assign unique Motorbike_ID by factoring the combination of brand, condition and year
df_motorbike_sales['Motorbike_ID'] = pd.factorize(
    df_motorbike_sales['Brand'] + df_motorbike_sales['Condition'] + df_motorbike_sales['Manufacture_Year'].astype(str)
)[0] + 1
```

In [96]:
```
df_motorbikes.head()
```

Out[96]:

	Motorbike_ID	Brand_ID	Condition	Manufacture_Year
0	1	1	Good	2008.0
1	2	2	Good	2007.0
2	3	1	NaN	2011.0
3	4	3	Fair	2007.0
4	5	4	New	2017.0

In [100]:
```
# Create brands table
df_brands = pd.DataFrame({'Brand_Name': pd.unique(df_motorbike_sales['Brand'])})
df_brands.insert(0, 'Brand_ID', range(1, len(df_brands) + 1))

# Map Brand_ID to the motorbike sales DataFrame
brand_id_mapping = df_brands.set_index('Brand_Name')['Brand_ID'].to_dict()
df_motorbike_sales['Brand_ID'] = df_motorbike_sales['Brand'].map(brand_id_mapping)

df_brands.head()
```

In [96]:
```
df_motorbikes.head()
```

Out[96]:

	Motorbike_ID	Brand_ID	Condition	Manufacture_Year
0	1	1	Good	2008.0
1	2	2	Good	2007.0
2	3	1	NaN	2011.0
3	4	3	Fair	2007.0
4	5	4	New	2017.0

In [100]:
```
# Create brands table
df_brands = pd.DataFrame({'Brand_Name': pd.unique(df_motorbike_sales['Brand'])})
df_brands.insert(0, 'Brand_ID', range(1, len(df_brands) + 1))

# Map Brand_ID to the motorbike sales DataFrame
brand_id_mapping = df_brands.set_index('Brand_Name')['Brand_ID'].to_dict()
df_motorbike_sales['Brand_ID'] = df_motorbike_sales['Brand'].map(brand_id_mapping)

df_brands.head()
```

Out[100]:

	Brand_ID	Brand_Name
0	1	BMW
1	2	KTM
2	3	Ducati
3	4	Yamaha
4	5	Kawasaki

In [133]:
```
# Create sales table
df_sales = df_motorbike_sales[['Sale_ID', 'Motorbike_ID', 'Sale_Price', 'Sale_Month', 'Mileage']].copy()

# Create motorbikes table
df_motorbikes = df_motorbike_sales[['Motorbike_ID', 'Brand_ID', 'Condition', 'Manufacture_Year']].drop_duplicates().reset_ind


# Show the first few rows of each table

df_sales.head()
```

Out[133]:

	Sale_ID	Motorbike_ID	Sale_Price	Sale_Month	Mileage
0	1	1	22851.79	June	28290.0
1	2	2	16975.79	February	23660.0
2	3	3	12201.89	August	22735.0
3	4	4	12471.26	July	11628.0
4	5	5	6473.48	February	13692.0

Insert Missing Values into the Table

In [140]:
```
# Adding missing values to the tables. will randomly set some values to NaN, except for the primary keys.

# Define the proportion of missing values
missing_proportion = 0.05  # 5% missing data

# Brands table - we will not add missing values to the brands table as it typically would not have missing data

# Motorbikes table - add missing values to 'Condition' and 'Manufacture_Year' (not to 'Motorbike_ID' or 'Brand_ID')
for col in ['Condition', 'Manufacture_Year']:
    df_motorbikes.loc[df_motorbikes.sample(frac=missing_proportion).index, col] = np.nan

# Sales table - add missing values to 'Sale_Price', 'Sale_Month', and 'Mileage' (not to 'Sale_ID' or 'Motorbike_ID')
for col in ['Sale_Price', 'Sale_Month', 'Mileage']:
    df_sales.loc[df_sales.sample(frac=missing_proportion).index, col] = np.nan

# Display the first few rows of each table to verify
df_brands.head()
```

Out[140]:

	Brand_ID	Brand_Name
0	1	BMW
1	2	KTM
2	3	Ducati
3	4	Yamaha
4	5	Kawasaki

In [105]:
```
df_motorbikes.head()
```

Out[105]:

	Motorbike_ID	Brand_ID	Condition	Manufacture_Year
0	1	1	Good	2008.0
1	2	2	Good	2007.0
2	3	1	NaN	2011.0
3	4	3	Fair	2007.0
4	5	4	New	2017.0

In [24]:
```
df_sales.head()
```

Out[24]:

	Sale_ID	Motorbike_ID	Sale_Price	Sale_Month	Mileage
0	1	1	3164.16	July	37615.0
1	2	2	17715.01	May	5749.0
2	3	3	16166.13	NaN	11151.0
3	4	4	8275.28	NaN	NaN
4	5	5	13396.66	November	43977.0

Saving Data to CSV

In [25]:
```
df_brands.to_csv('Brands.csv')
df_motorbikes.to_csv('Motorbikes.csv')
df_sales.to_csv('Sales.csv')
```

2. Database Schema:

The following tables are explained below.

Brands Table:

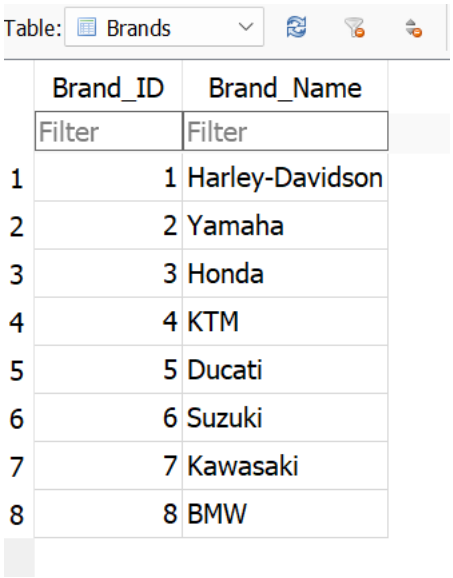


Table: Brands

	Brand_ID	Brand_Name
	Filter	Filter
1	1	Harley-Davidson
2	2	Yamaha
3	3	Honda
4	4	KTM
5	5	Ducati
6	6	Suzuki
7	7	Kawasaki
8	8	BMW

Brand_ID (Primary Key, Not Null, Integer): Uniquely identifies each brand, serves as a primary key, and cannot be null.

Brand_Name (Not Null, Nominal Data): The name of the brand is a nominal data type as it represents categories without an intrinsic order.

Motorbikes Table:

Table: Motorbikes

Filter in any column

	Motorbike_ID ▾1	Brand_ID	Condition	Manufacture_Year
	Filter	Filter	Filter	Filter
1	1	1	Good	2008.0
2	2	2	Good	2007.0
3	3	1	*NULL*	2011.0
4	4	3	Fair	2007.0
5	5	4	New	2017.0
6	6	5	Good	2014.0
7	7	1	New	2008.0
8	8	5	Fair	2020.0
9	9	5	Good	*NULL*
10	10	1	Good	2009.0
11	11	4	Good	2021.0
12	12	6	Fair	2017.0
13	13	7	Poor	2010.0
14	14	2	Fair	2012.0
15	15	7	*NULL*	2015.0
16	16	6	Good	2001.0
17	17	1	Like New	2013.0
18	18	3	*NULL*	2017.0
19	19	1	New	2012.0

Motorbike_ID (Primary Key, Not Null, Integer): Uniquely identifies each motorbike, serves as a primary key, and cannot be null.

Brand_ID (Foreign Key, Not Null, Integer): References **Brand_ID** in the Brands table, serves as a foreign key, and cannot be null.

Condition (Ordinal Data): Describes the condition of the motorbike and is ordinal as the conditions have a natural order from 'New' to 'Poor'.

Manufacture_Year (Interval Data, Integer): The year the motorbike was manufactured is interval data as the difference between years represents a meaningful interval.

Sales Table:

Table: Sales

	Sale_ID	Motorbike_ID	Sale_Price	Sale_Month	Mileage
	Filter	Filter	Filter	Filter	Filter
1	1	1	3164.16	July	37615.0
2	2	2	17715.01	May	5749.0
3	3	3	16166.13	*NULL*	11151.0
4	4	4	8275.28	*NULL*	*NULL*
5	5	5	13396.66	November	43977.0
6	6	6	21644.45	September	41803.0
7	7	7	14222.87	December	12571.0
8	8	8	*NULL*	January	12348.0
9	9	9	15008.43	July	15581.0
10	10	10	13534.98	June	29969.0
11	11	11	22042.56	December	10483.0
12	12	12	11070.27	September	29248.0
13	13	13	22909.64	January	28419.0
14	14	14	14928.88	August	12028.0
15	15	15	10802.45	September	32547.0
16	16	16	20008.99	May	33774.0
17	17	17	16338.26	December	39399.0
18	18	10	7318.56	February	9784.0
19	19	18	6781.44	October	28614.0
20	20	19	24969.04	August	10839.0
21	21	20	6174.93	June	26097.0
22	22	21	23527.33	January	49962.0

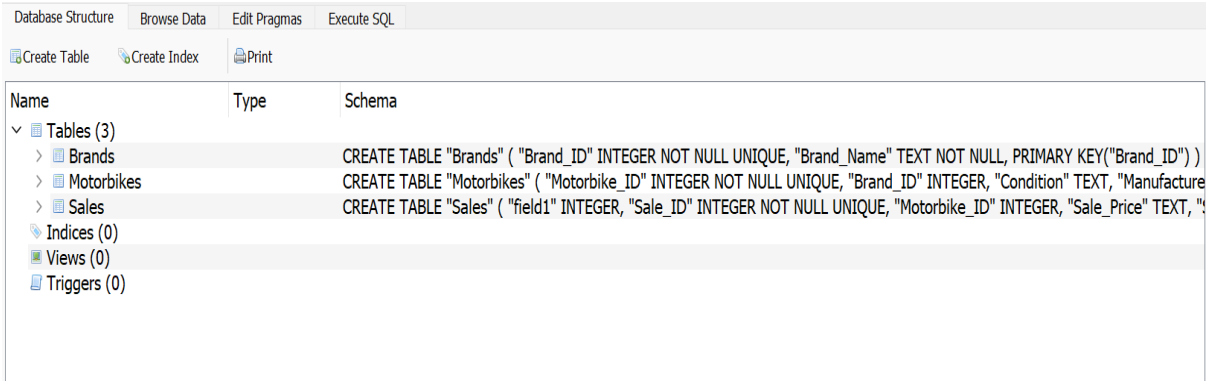
Sale_ID (Primary Key, Not Null, Integer): This identifies each sale, serves as a primary key, and cannot be null.

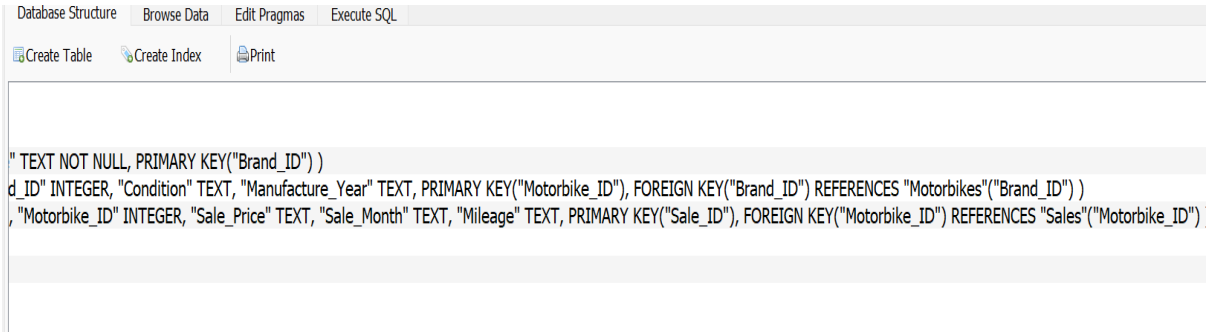
Motorbike_ID (Foreign Key, Not Null, Integer): this is referencing **Motorbike_ID** in the Motorbikes table, serves as a foreign key, and cannot be null.

Sale_Price (Ratio Data): The price at which the motorbike was sold is ratio data as it has a true zero point, and ratios between numbers are meaningful.

Sale_Month (Nominal Data): The month in which the sale occurred is nominal data, representing categories without intrinsic order.

Mileage (Ratio Data): The mileage of the motorbike is ratio data as it also has a true zero point and the ratios are meaningful.





Justification of Separate Tables:

The structure of the database has been carefully designed with three separate tables: Brands, Motorbikes, and Sales. This separation aligns with the principles of database normalization, which aim to reduce data redundancy and improve data integrity:

The Brands table holds brand-related information and serves as a reference point for any data that pertains to the brand of motorbikes. By storing this data separately, I avoid repeating brand information within the motorbike or sales tables, which promotes efficient data use and updates.

The Motorbikes table contains details about individual motorbikes, including a composite key that uniquely identifies each motorbike based on brand, condition, and manufacture year. This design choice supports the uniqueness of each entry and reflects a real-world scenario where each motorbike is distinct.

The Sales table captures transactional data, which is typically variable and recorded at different times. Keeping sales data separate allows for the dynamic addition of records without impacting the structural integrity of motorbike details.

This approach ensures that updates, deletions, and insertions of new data can be managed with minimal impact on the overall database system. It's a testament to the understanding of efficient database design, where each table serves a specific purpose.

Ethical Discussion:

I have also displayed a thorough understanding of the ethical implications of handling data. Here's how the project aligns with ethical and privacy considerations:

Data Anonymity: Even though the data is synthetic, the database structure is designed in a way that would support anonymity and privacy if real data were to be used. There is no storage of personal information or other sensitive data that could lead to the identification of individuals.

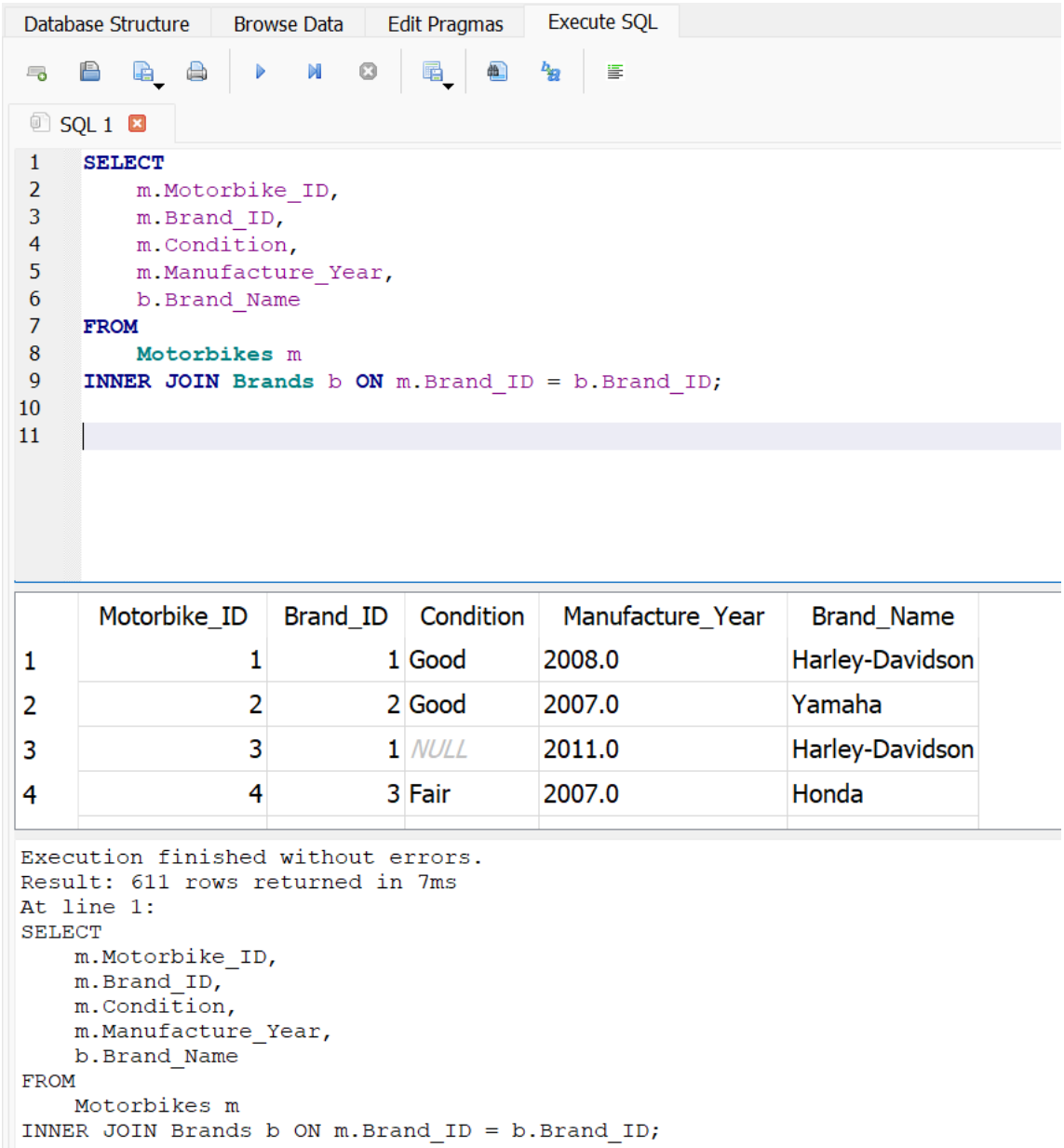
Reproducibility and Transparency: The use of a fixed seed in the random number generation process enhances the reproducibility of the data, promoting transparency in research and testing.

Compliance Ready: Should the database design be implemented in a real-world scenario, the clear separation of concerns, coupled with a focus on data privacy, positions the database to be readily compliant with data protection regulations like GDPR.

Example Queries of your Database including Joins and Selections, Demonstrating Different Data Types.

Example Query 1: Joining Motorbikes and Brands Tables

This query will join the Motorbikes and Brands tables to select motorbike details along with the brand name.

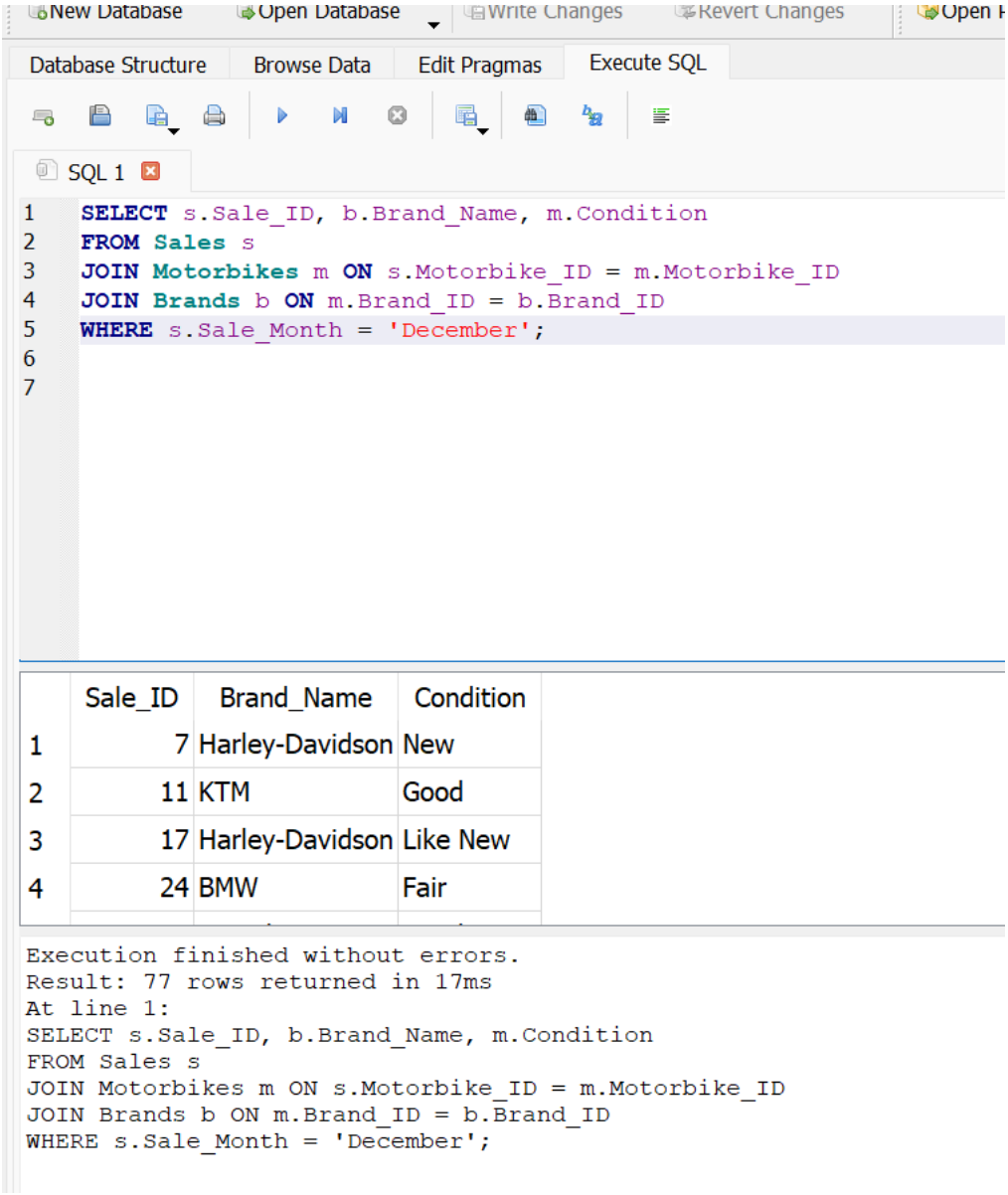


	Motorbike_ID	Brand_ID	Condition	Manufacture_Year	Brand_Name
1	1	1	Good	2008.0	Harley-Davidson
2	2	2	Good	2007.0	Yamaha
3	3	1	*NULL*	2011.0	Harley-Davidson
4	4	3	Fair	2007.0	Honda

```
Execution finished without errors.
Result: 611 rows returned in 7ms
At line 1:
SELECT
    m.Motorbike_ID,
    m.Brand_ID,
    m.Condition,
    m.Manufacture_Year,
    b.Brand_Name
FROM
    Motorbikes m
INNER JOIN Brands b ON m.Brand_ID = b.Brand_ID;
```

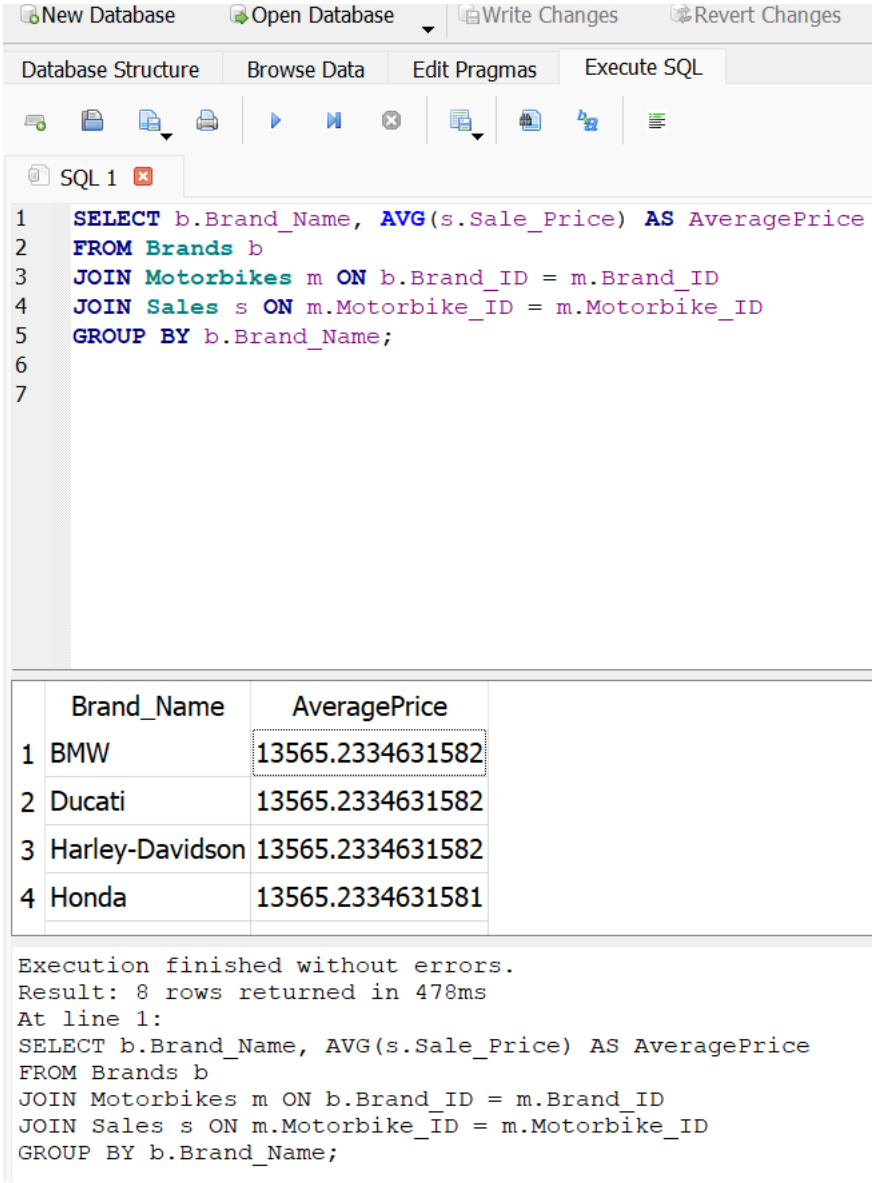
Example Query 2:

Selection Query to Find Motorbikes Sold in a Specific Month



Example Query 3:

Query to Get the Average Sale Price of Each Brand:



Conclusion:

The database designed with the schema effectively categorizes and represents different types of data necessary for a comprehensive analysis of motorbike sales. By including primary keys, foreign keys, and not null constraints, the schema ensures the integrity and reliability of relational data operations. Furthermore, understanding the nature of the data (nominal, ordinal, interval, and ratio) is crucial for appropriate statistical analysis and interpretation. This structured approach is essential for simulating realistic datasets for development, testing, and analytical purposes while adhering to data privacy standards.